ARBIX AI SOLUTIONS

Full-Stack AI Engineer

Round 1 Practical Exercise Spec

Time-box	Submission	Tech stack	What we value
90 minutes	GitHub repo link only	Python backend + React UI	Correctness, validation, logging, tests, judgement

Purpose: This exercise simulates a small slice of the kind of work you would do at Arbix AI, building a clean backend service, connecting it to a simple frontend, and showing good engineering judgement around validation, logging, testing, and clarity.

Exercise Specification

Allowed resources: Google, official documentation, open-source libraries, and LLM tools such as ChatGPT, Claude and Copilot are allowed, with disclosure. **Not allowed:** getting help from another person, outsourcing, or submitting a pre-built project with only minor edits.

Overview: This practical exercise is designed to simulate a small slice of the kind of work you would do at Arbix AI: building a clean backend service, connecting it to a simple frontend, and showing good engineering judgement around validation, logging, testing, and clarity.

Time-box: 90 minutes total. Please do not exceed this time.

Submission: Submit one GitHub repository link only.

Goal

- Build a small end-to-end scoring application with a Python backend API and a minimal React frontend.
- Show clear input validation, basic explainability, simple audit logging, and basic tests.
- We care more about correctness, structure, and judgement than visual polish.

Part A - Backend (Python)

Use FastAPI or Flask.

1. **Create a POST /score endpoint.**

The endpoint should accept JSON input with the following fields:

Field	Type / rule	Notes
land_area_acres	number; > 0	Land area in acres; must be positive
crop_type	non-empty string	Any non-empty crop label
repayment_history_score	number; 0–100	Inclusive range 0 to 100
annual_income_band	string; enum	<2L, 2–5L, 5–10L, >10L

Expected response: Return JSON with request_id, score, reason_codes, and timestamp.

Field	Expectation
request_id	UUID or unique ID
score	Number from 0 to 100
reason_codes	Array of exactly 3 short strings, e.g. good_repayment, small_landholding, mid_income_band
timestamp	ISO 8601 string

Scoring logic: The scoring logic can be simple and rule-based. No ML training is required. We are evaluating your engineering structure and judgement, not model complexity.

2. **Validation and error handling.**

- Implement clear validation and return useful 4xx responses for invalid input.
- Examples: wrong data type, missing field, empty crop type, invalid income band, negative or zero land area, or repayment_history_score outside 0–100.
- Use appropriate status codes such as 400 or 422.

3. **Audit logging.**

- Log every scoring request with request_id, timestamp, key input fields used for scoring, score, and reason_codes.
- Structured console logging or file-based logging is acceptable.
- Please avoid logging anything you consider sensitive.

4. **Tests.**

- Add at least 2-unit tests: 1 happy-path test and 1 validation/error-path test.

Part B — Frontend (React)

- Build a minimal React UI with a form for the 4 input fields and a button to submit to the backend /score endpoint.
- Display the score and reason codes.
- Handle loading state, validation errors, and backend errors gracefully.
- React with JavaScript or TypeScript is fine. Basic styling is enough. UI polish is not the focus.

Database

Database usage is optional. If you choose to add persistence as a bonus, SQLite is preferred for simplicity. Postgres is also fine if you already have it available. There are no extra marks for adding a database unless it is implemented cleanly within the time-box.

Time-box proof (required)

1. Create a fresh repository or a clearly separate new branch for this exercise.
2. Make your first commit at your chosen start time.
3. Make your final commit near the end of the 90-minute window.
4. Include at least 4 meaningful commits. Please avoid one large dump commit at the end.

In README.md, include:

- start time (IST)
- end time (IST)
- approximate total time spent
- what you completed
- what you skipped, if anything, and why

LLM/tool usage (allowed, but disclosure required)

You may use LLMs and coding tools. We care about your judgement and your ability to review, adapt, and validate output.

In README.md, include a short section covering:

- which tool(s) you used
- where/how you used them
- what you personally checked, changed, or corrected

Also add a file named LLM_NOTES.md containing:

- 2 to 5 example prompts you used (paraphrased prompts are fine)
- one example of something you improved or corrected after reviewing tool output

Optional: If you are comfortable, you may include an exported llm_chat.pdf with personal or sensitive information redacted. This is optional and not required.

Suggested project structure

/backend
/frontend
README.md
LLM_NOTES.md

Deliverables (required)

- backend code
- frontend code
- README.md with run steps, design choices / tradeoffs, time-box details, LLM/tool disclosure, and what you would improve next if given 2 more hours
- LLM_NOTES.md
- tests

Bonus items (optional)

- Dockerfile or docker-compose
- linting / formatting setup
- a simple drift-check endpoint using a toy PSI or similar idea
- lightweight persistence

Please do not sacrifice the core exercise for bonus items.

Evaluation criteria

- working solution end to end
- code clarity and structure
- validation and error handling
- audit logging approach
- basic tests
- frontend integration
- practicality of your implementation choices
- your ability to explain and extend the solution in the live review

Important note: Correctness, structure, and engineering judgement matter more than visual polish.

What we expect

- We do not expect a perfect production system in 90 minutes.
- We do expect clean judgement, working core functionality, reasonable tradeoffs, clear communication in the README, and evidence that you understand what you built.